

pline-vec

Jasper

May 15, 2025

Contents

1	Introduction	1
2	Examples	1
2.1	Parametric mappings, and the pline-vec math libraries	1
2.1.1	Parametric mappings	1
2.1.2	The pline-vec math libraries	2
2.2	Setting the main orientation	4
2.2.1	Working with sub-orientations	4
2.3	Rendering parametric curves	4
2.4	Rendering parametric surfaces	4
2.5	Rendering parametric solids	4
2.6	Rendering parametric curves, surfaces and solids all at once	4
2.6.1	The problem with tangency	4
2.7	Clipping planes by a rectangular prism	4
2.8	Intersecting planes — in progress	4

1 Introduction

pline-vec is a 3D toolkit for TikZ.

2 Examples

2.1 Parametric mappings, and the pline-vec math libraries

2.1.1 Parametric mappings

A parametric mapping is a mathematical rule which can accept any number of inputs, and produces any number of outputs, all according to some mathematical formula. For example, A curve in $\mathbb{R}^3$ is defined by a parametric function which accepts one input and delivers three outputs. Similarly, a surface in $\mathbb{R}^3$ is defined by a parametric function which accepts two inputs, and delivers three outputs. The parametric equation for a solid accepts three inputs and delivers

three outputs. Parametric mappings are useful for traversing many common mathematical objects. For example, they can be used to navigate the sphere — which enables us to produce direction vectors in any direction, based on longitude and latitude.

2.1.2 The pline-vec math libraries

There are two math libraries for pline-vec. The main one is `math.lua`, which defines several useful 3D math functions. In the lua library, to avoid naming conflicts, they are prefixed with “`pv_`”. Similarly, they are prefixed with “`pv`” in the `pgfmath` library. These are the commands from the lua library:

- `pv_cross_product_x(u,v)`
Delivers the x component of the cross product of two vectors.
- `pv_cross_product_y(u,v)`
Delivers the y component of the cross product of two vectors.
- `pv_cross_product_z(u,v)`
Delivers the z component of the cross product of two vectors.
- `pv_dot_product(u,v)`
Delivers the dot product of two vectors.
- `pv_norm(u)`
Delivers the norm of a vector
- `pv_ZYZ_rotation_matrix_x(angles,vector)`
Delivers the x component of the ZYZ totation matrix of two vectors.
- `pv_ZYZ_rotation_matrix_y(angles,vector)`
Delivers the y component of the ZYZ totation matrix of two vectors.
- `pv_ZYZ_rotation_matrix_z(angles,vector)`
Delivers the z component of the ZYZ totation matrix of two vectors.
- `pv_sphere_x(longitude,latitude)`
Delivers the x component of the sphere, based on longitude and latitude.
- `pv_sphere_y(longitude,latitude)`
Delivers the y component of the sphere, based on longitude and latitude.
- `pv_sphere_z(longitude,latitude)`
Delivers the z component of the sphere, based on longitude and latitude.

The `pgfmath` library is also lua-based, though it is done internally.

- `pvcrossproductx`
x-component of the cross product of u and v, #1 - u1, #2 - u2, #3 - u3, #4 - v1, #5 - v2, #6 - v3
- `pvcrossproducty`
y-component of the cross product of u and v, #1 - u1, #2 - u2, #3 - u3, #4 - v1, #5 - v2, #6 - v3
- `pvcrossproductz`
z-component of the cross product of u and v, #1 - u1, #2 - u2, #3 - u3, #4 - v1, #5 - v2, #6 - v3
- `pvdotproduct`
dot product of u and v, #1 - u1, #2 - u2, #3 - u3, #4 - v1, #5 - v2, #6 - v3
- `pvnorm`
norm product of u #1 - u1, #2 - u2, #3 - u3
- `pvtransformrotmainx`
x-component of ZYZ rotation matrix on a point #1 - alpha, #2 - beta, #3 - gamma, #4 - u1, #5 - u2, #6 - u3
- `pvtransformrotmainy`
y-component of ZYZ rotation matrix on a point #1 - alpha, #2 - beta, #3 - gamma, #4 - u1, #5 - u2, #6 - u3
- `pvtransformrotmainz`
z-component of ZYZ rotation matrix on a point #1 - alpha, #2 - beta, #3 - gamma, #4 - u1, #5 - u2, #6 - u3
- `pvspherex`
x-component of the sphere, given longitude and latitude #1 - longitude, #2 - latitude
- `pvspherey`
y-component of the sphere, given longitude and latitude #1 - longitude, #2 - latitude
- `pvspherez`
z-component of the sphere, given longitude and latitude #1 - longitude, #2 - latitude

- 2.2 Setting the main orientation
 - 2.2.1 Working with sub-orientations
- 2.3 Rendering parametric curves
- 2.4 Rendering parametric surfaces
- 2.5 Rendering parametric solids
- 2.6 Rendering parametric curves, surfaces and solids all at once
 - 2.6.1 The problem with tangency
- 2.7 Clipping planes by a rectangular prism
- 2.8 Intersecting planes — in progress